American International University-Bangladesh
Department of Computer Science

CSC4164

Advanced Programming

.NET / ASP.NET Core MVC

Lab 2

Routing & Razor Views

Course Code	CSC4164
Course Title	Advanced Programming (.NET / ASP.NET Core MVC)
Lab Session	Lab 2 — Week 2
Instructor	Jahid Hassan
Duration	2 Hours 20 Minutes (140 minutes)
Work Mode	Individual
Lab Task	None this session
Prerequisite	Pre-Lab Assignment completed — FirstMVCApp project working

1. Lab Objectives

By the end of this lab session, you will be able to:

- Configure conventional and attribute-based routing in ASP.NET Core MVC
- Define routes with parameters, optional segments, default values, and constraints
- Write Razor syntax to output variables, use conditionals, and render loops
- Pass data from a controller to a view using ViewBag and ViewData
- Build a shared layout with navigation and apply it across multiple views
- Define and render named sections within a layout

2. Prerequisites

- Pre-Lab Assignment completed — FirstMVCApp project created and running
- Theory class Week 2 attended — Module 2 (Routing) and Module 3 (Razor Views)

> **Warning:** Start of Lab Check: Your instructor will ask you to run your FirstMVCApp from the Pre-Lab Assignment in the first 10 minutes. If it does not run, raise your hand immediately.

3. Theory Recap

3.1 Routing

Routing maps an incoming HTTP request URL to a specific controller action. ASP.NET Core MVC supports two styles:

Style	Where Defined	Example
Conventional	Program.cs — applies globally	/Student/Details/5 → StudentController.Details(5)
Attribute	Directly on the controller using [Route]	[Route("students/{id}")] on the action method

The default conventional route pattern in Program.cs is:

```
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

// {controller=Home}  =>  defaults to HomeController if not specified
// {action=Index}     =>  defaults to Index() if not specified
// {id?}              =>  optional route parameter
```

3.2 Razor Syntax

Razor allows you to mix C# with plain HTML using the @ symbol. The rule is simple: write plain HTML wherever possible, and use Razor only where dynamic content is needed.

Syntax	Purpose	Example
@variable	Output a C# variable inline in HTML	@ViewBag.Title
@{ }	C# code block with no output	@{ var count = 5; }
@if ( ) { }	Conditional rendering	@if (Model != null) { Found }
@foreach ( ) { }	Loop over a collection	@foreach (var s in Model) { @s }

4. Lab Activities

> **Warning:** Important: Type all code manually. Do not copy and paste from this manual. Typing the code builds muscle memory, helps you spot errors, and improves your understanding of the syntax. Students found copy-pasting will be asked to retype.

Task 1 Startup Check — Verify Pre-Lab Project (10 min)

Before starting new work, confirm your Pre-Lab project is working correctly.

1. Open FirstMVCApp in Visual Studio
2. Press F5 to run the application
3. Navigate to /Student/Index and confirm the page loads
4. Navigate to /Student/Details/3 and confirm the student ID is displayed
5. If anything is broken, raise your hand — do not proceed until your base project works

Task 2 Conventional Routing — Parameters, Defaults & Constraints (25 min)

You will add a second route and test how parameters, default values, and constraints work.

Step 1 — Add a second route in Program.cs

Open Program.cs and add the product route above the default route:

```
app.MapControllerRoute(
    name: "product",
    pattern: "products/{category}/{id:int}",
    defaults: new { controller = "Product", action = "List" });

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

Step 2 — Create ProductController

Right-click Controllers → Add → Controller → MVC Controller — Empty. Name it ProductController and type:

```
public class ProductController : Controller
{
    public IActionResult List(string category, int id)
    {
        ViewBag.Category = category;
        ViewBag.Id = id;
        return View();
    }
}
```

Step 3 — Create the List view

Create Views/Product/List.cshtml. Write plain HTML for structure, Razor only for dynamic values:

```
@{
    ViewData["Title"] = "Product List";
}

<h2>Product List</h2>
<p>Category: <strong>@ViewBag.Category</strong></p>
<p>Product ID: <strong>@ViewBag.Id</strong></p>
```

Step 4 — Test the route

6. Run the application
7. Navigate to /products/electronics/5 — confirm Category and ID display correctly
8. Navigate to /products/books/abc — confirm 404 response (the :int constraint rejects non-numeric IDs)

Task 3 Attribute Routing (25 min)

You will create a controller that uses attribute routing instead of conventional routing.

Step 1 — Create CourseController with attribute routes

Add a new empty MVC controller named CourseController and type the following:

```
using Microsoft.AspNetCore.Mvc;

namespace FirstMVCApp.Controllers
{
    [Route("courses")]
    public class CourseController : Controller
    {
        // Accessible at: /courses
        [Route("")]
        public IActionResult Index()
        {
            ViewBag.Title = "All Courses";
            return View();
        }

        // Accessible at: /courses/details/5
        [Route("details/{id:int}")]
        public IActionResult Details(int id)
        {
            ViewBag.CourseId = id;
            return View();
        }

        // Accessible at: /courses/search?keyword=mvc
        [Route("search")]
        public IActionResult Search(string keyword)
        {
```

```
            ViewBag.Keyword = keyword;
            return View();
        }
    }
}
```

Step 2 — Create views for each action

Create Views/Course/ folder and add Index.cshtml, Details.cshtml, and Search.cshtml. Use plain HTML with Razor only for dynamic values. For example, Index.cshtml:

```
@{
    ViewData["Title"] = "Courses";
}

<h2>@ViewBag.Title</h2>
<p>Welcome to the courses section.</p>
<ul>
    <li><a href="/courses/details/1">Course 1</a></li>
    <li><a href="/courses/details/2">Course 2</a></li>
</ul>
```

Step 3 — Test all routes

9. Navigate to /courses — should show the Index view
10. Navigate to /courses/details/10 — should show CourseId = 10
11. Navigate to /courses/search?keyword=mvc — should show Keyword = mvc

> **Note:** Notice the difference: with attribute routing the URL is /courses/details/10, not /Course/Details/10. The [Route] attribute fully controls the URL shape.

Task 4 Razor Syntax — Loops, Conditions, ViewBag & ViewData (25 min)

You will update the Student module to display a dynamic list using Razor syntax with plain HTML structure.

Step 1 — Update StudentController.Index to pass a list

```
public IActionResult Index()
{
    ViewBag.Title = "Student List";
    ViewData["SubTitle"] = "Enrolled students this semester";

    var students = new List<string>
    {
        "Alice Rahman", "Bob Hasan", "Carol Akter", "David Khan"
    };

    return View(students);
}
```

Step 2 — Update Views/Student/Index.cshtml

Write the HTML table structure first, then add Razor only where dynamic content is needed:

```
@model List<string>

@{
    ViewData["Title"] = "Students";
}

<h2>@ViewBag.Title</h2>
<p><em>@ViewData["SubTitle"]</em></p>

@if (Model != null && Model.Count > 0)
{
    <table class="table table-striped">
        <thead>
        <tr>
            <th>#</th>
            <th>Name</th>
        </tr>
        </thead>
        <tbody>
        @{ int i = 1; }
        @foreach (var student in Model)
        {
            <tr>
                <td>@i</td>
                <td>@student</td>
            </tr>
            i++;
        }
        </tbody>
    </table>
}
else
{
    <p>No students found.</p>
}
```

12. Run the application and navigate to /Student/Index
13. Confirm the table renders all four students with row numbers
14. Temporarily change the list to new List<string>() in the controller and confirm the else branch renders
15. Restore the list before moving on

Task 5 Shared Layout & Sections (30 min)

You will update the shared layout to include a navigation bar and define a named section that individual views can fill.

Step 1 — Update _Layout.cshtml navigation

Open Views/Shared/_Layout.cshtml and update the navigation area inside the header tag:

```
<ul class="navbar-nav flex-grow-1">
    <li class="nav-item">
        <a class="nav-link text-dark" asp-controller="Home" asp-
action="Index">Home</a>
    </li>
    <li class="nav-item">
        <a class="nav-link text-dark" asp-controller="Student" asp-
action="Index">Students</a>
    </li>
    <li class="nav-item">
        <a class="nav-link text-dark" asp-controller="Course" asp-
action="Index">Courses</a>
    </li>
    <li class="nav-item">
        <a class="nav-link text-dark" asp-controller="Product" asp-action="List"
asp-route-category="all" asp-route-id="0">Products</a>
    </li>
</ul>
```

Step 2 — Add a named section to the layout

Below the nav element and above @RenderBody(), add the following:

```
<!-- Optional section — views can fill this or leave it empty -->
@RenderSection("PageActions", required: false)

@RenderBody()
```

Step 3 — Use the PageActions section in Student Index

Add the following at the top of Views/Student/Index.cshtml:

```
@section PageActions {
    <div style="background: #f0f0f0; padding: 8px; margin-bottom: 10px;">
        <a asp-controller="Student" asp-action="Details" asp-route-id="1">View
Student 1</a>
    </div>
}
```

Step 4 — Test the layout and section

16. Run the application and confirm the navigation bar appears on all pages
17. Navigate to /Student/Index and confirm the PageActions section renders above the content
18. Navigate to /courses and confirm the PageActions section does not appear there

Task 6 Route Testing — Test at Least 7 Route Patterns (10 min)

Test the following URLs in your browser and record your observation for each:

URL to Test	Expected Result	Your Observation
/	Home/Index (default route)	
/Student	Student/Index (default action)	
/Student/Details/7	Student Details with ID = 7	
/courses	Course Index via attribute route	
/courses/details/99	Course Details with ID = 99	
/products/electronics/3	Product List: electronics, ID = 3	
/products/books/xyz	404 — :int constraint rejects non-integer	
/courses/search?keyword=dotnet	Search view with keyword = dotnet	

> **Tip:** Fill in the Your Observation column as you test. This trains you to read route patterns and predict behaviour — a common quiz topic.

5. Common Errors and Solutions

Error	Likely Cause	Solution
404 on a route	Controller or action name mismatch in URL	Controller name in URL omits the 'Controller' suffix; check spelling
Ambiguous route exception	Two routes match the same URL pattern	More specific routes must be registered above less specific ones in Program.cs
@model type mismatch error	View model type does not match what controller sends	Ensure controller passes List<string> and view declares @model List<string>
Section not rendering in layout	Section name typo or required: true when view omits it	Check names match exactly; set required: false for optional sections
Navigation links show wrong URL	asp-route parameter name does not match action parameter	Route parameter names in tag helpers must match action method parameter names

6. Completion Checklist

#	Checklist Item	Done
1	Pre-Lab project verified and running at session start	☐
2	ProductController created with conventional route using category and id parameters	☐
3	Route constraint :int tested — non-integer ID confirmed to return 404	☐
4	CourseController created with attribute routing on all three actions	☐
5	All three course routes tested: /courses, /courses/details/id, /courses/search?keyword=	☐
6	StudentController.Index updated to pass a List<string> model to the view	☐
7	Razor view updated with @model, @foreach, @if, ViewBag and ViewData	☐
8	Shared layout updated with navigation bar linking all controllers	☐
9	Named section PageActions defined in layout and used in Student Index view	☐
10	Confirmed PageActions section does not appear on pages that do not define it	☐
11	All 8 route patterns in Task 6 tested and observations recorded	☐

7. Self-Assessment Questions

Review these questions before leaving. They may appear in Quiz 1.

- What is the difference between conventional routing and attribute routing? When would you use each?
- What does the {id?} token mean in a route pattern?

- What is the difference between ViewBag and ViewData?
- What does @RenderBody() do in a layout file?
- What does required: false mean in @RenderSection?
- If a route has the constraint {id:int}, what happens when a non-integer value is passed?

8. Further Reading

- Routing in ASP.NET Core: https://learn.microsoft.com/aspnet/core/fundamentals/routing
- Razor syntax reference: https://learn.microsoft.com/aspnet/core/mvc/views/razor
- Layout in ASP.NET Core MVC: https://learn.microsoft.com/aspnet/core/mvc/views/layout

End of Lab 2 — Lab 3 covers Model Binding and Validation.